CONCEPT NOTE · INSTRUMENTATION & DSP

How Coherence Turns a DAQ Card Into a Lock-In Amplifier

A from-first-principles walkthrough for anyone joining this project cold: what a 4431/4461 actually does, what a lock-in amplifier is and why you'd want one, and the three ways this codebase can demodulate your signals — the classic per-channel way, the FFT way, and the streaming way — with the actual trade-offs, limits, and failure modes of each.

Hardware: NI USB-4431 / PXIe-4461 (DSA family) **Software:** Coherence

Audience: new to lock-in DSP, new to this codebase, or both

CHAPTER 1

What a DAQ card actually does

Before anything about lock-ins: what is a 4431 or 4461, mechanically?

A data acquisition (DAQ) card has two independent jobs. **AI** (analog input) turns a voltage on a wire into a stream of numbers. **AO** (analog output) does the reverse — turns a stream of numbers into a voltage on a wire. Everything else in this document is built on top of just those two operations, done fast and done together.

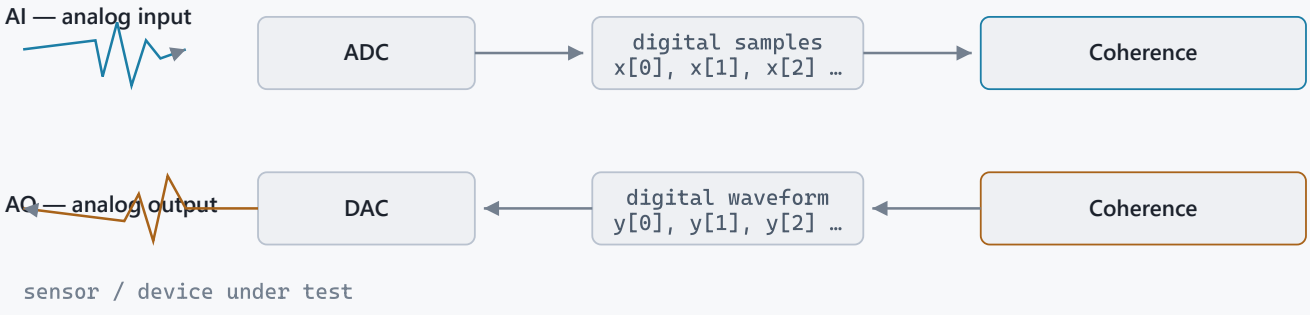


FIG 1.1 The only two primitives on the card: a voltage becomes numbers (AI), or numbers become a voltage (AO). Everything in this document is these two operations, run continuously and read/written in step.

Sample rate and Nyquist

The ADC doesn't watch the signal continuously — it takes a snapshot at a fixed rate, `fs` (samples/second, i.e. Hz). That single number sets a hard ceiling on what you can measure: the **Nyquist frequency**, `fs/2`. Any real signal component above that folds back (aliases) onto a lower, wrong frequency and is indistinguishable from a genuine low-frequency signal. A 4461 sampling at 204.8 kS/s can represent frequencies up to 102.4 kHz — not a byte higher.

Simultaneous sampling — why the 4431/4461 specifically

Cheaper DAQ cards multiplex: one shared ADC steps through each channel in turn, so channel 2's sample is captured a few microseconds after channel 1's. The 4431 and 4461 are **DSA** (Dynamic Signal Acquisition) cards — every channel has its own delta-sigma converter, so all channels are captured at genuinely the same instant. That matters enormously for a lock-in: a lock-in's whole output is a *phase* measurement, and a few microseconds of inter-channel skew is a real phase error at any decent frequency. Simultaneous sampling is what makes trustworthy multichannel phase measurement possible at all — keep this in mind for Chapter 7, where it comes back as the reason multiple cards need extra wiring to stay in step with each other.

> **Delta-sigma also means something else:** these converters run an internal oversampling/decimation filter, which carries a few samples of settling state at all times. That's invisible on one card, but it's exactly why two DSA cards can't just share a clock and call it synchronized — more in Chapter 7.

What a lock-in amplifier does, and why

The problem a lock-in solves: finding a signal you already know the frequency of, buried under noise much bigger than the signal itself.

Say you drive a sensor with a known tone at frequency f, and read its response. The response you actually measure is that small signal plus whatever noise, interference, and drift happens to be sitting on the wire — often orders of magnitude larger. You cannot simply "measure the voltage." You need to ask a much narrower question: *of everything on this wire, how much of it is oscillating at exactly f, in phase with my reference?* That's a lock-in.

Synchronous demodulation

The mechanism is a multiply-then-average. Multiply the incoming signal by a reference oscillator running at the same frequency f, then average (low-pass filter) the product. Anything in the input that shares the reference's frequency and phase turns into a steady DC term after multiplication; everything else — noise, interference, anything at a different frequency — turns into something that oscillates, and averages toward zero the longer you average. The averaging window is the classic lock-in **time constant** dial: longer window, more noise rejected, slower to respond.

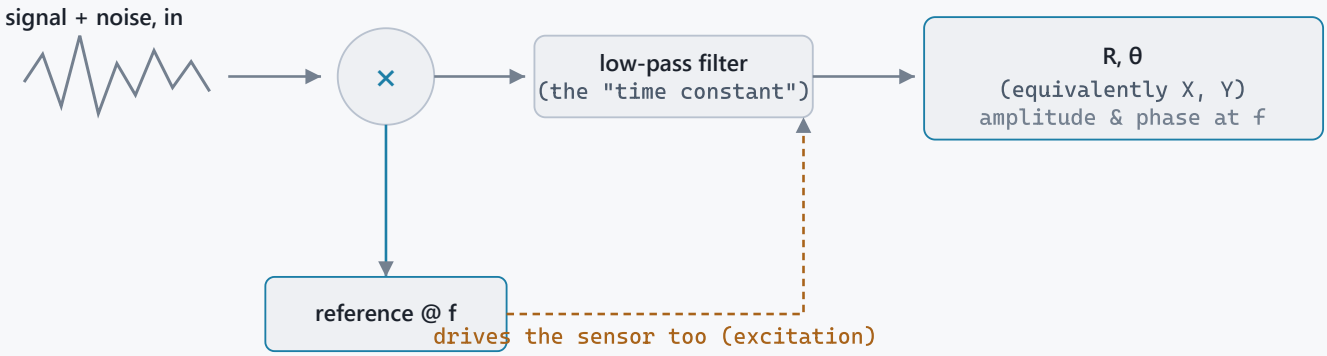


FIG 2.1 Synchronous demodulation: multiply by a reference at the frequency of interest, then low-pass filter. Everything not at that frequency and phase averages away. The same reference typically also drives the excitation, so demodulator and stimulus share one clock.

R/θ and X/Y

Because a real signal has both amplitude and phase, the reference is really two references 90° apart (a cosine and a sine, or equivalently one complex exponential). That gives two outputs, X (in-phase) and Y (quadrature), from which amplitude and phase follow directly:

$R = 2 \cdot \sqrt{(X^2 + Y^2)}$	amplitude, same units as the input
$\theta = \operatorname{atan2}(Y, X)$	phase relative to the reference

Every engine described in this document — the traditional per-channel approach, Coherence's FFT engine, and Coherence's streaming engine — computes exactly this. They differ only in *how* the multiply and the filtering actually happen in software, which changes speed, cost, and a few hard requirements, but never the underlying measurement.

CHAPTER 3

Building a multichannel lock-in out of a DAQ card

A 4431/4461 gives you several AI channels and, on the 4461, AO channels too. Put a reference generator on AO and a demodulator on AI, in software, and the card *is* a lock-in — no dedicated lock-in instrument required.

Scale that to many sensors and the question becomes: how do N channels share the work? This is where the three approaches in this codebase diverge. The original way — the way the LabVIEW-based predecessor to this project works, and the way most bench lock-ins work internally — is the direct one:

The traditional way: one demodulator chain per channel

Give every channel its own reference oscillator, its own multiplier, its own low-pass filter. It works, and it's conceptually the simplest thing to build. But the cost is linear in channel count: 12 channels means 12 independent mixer+filter chains, each doing real work on every single incoming sample.

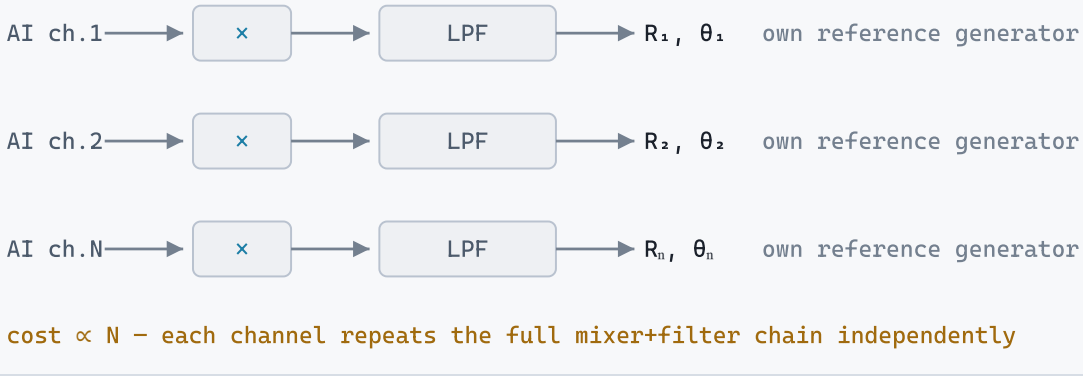


FIG 3.1 The traditional/LabVIEW architecture: N independent chains. Simple, and completely general — but N× the per-sample computation of a single channel.

Coherence keeps this same underlying math but restructures *who does the multiply-and-filter* to avoid paying the full cost N times. The two ways it does that — FFT and streaming — are the rest of this document.

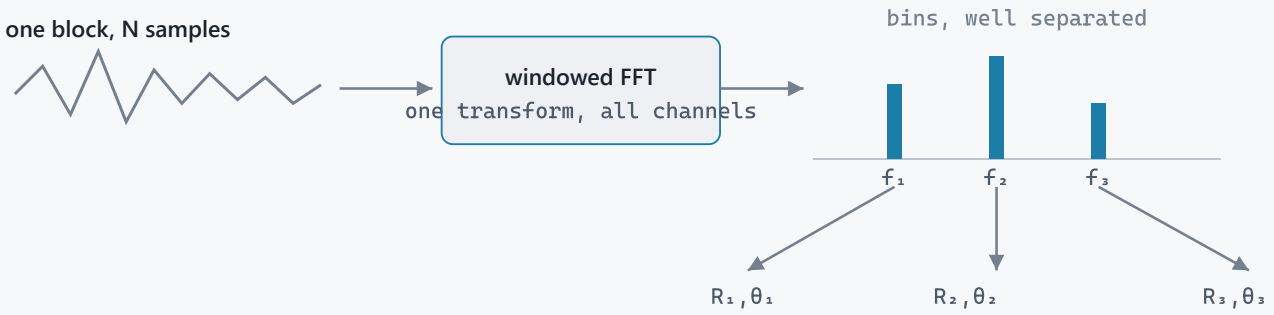
CHAPTER 4

The FFT approach

Coherence's original engine. The insight: a single DFT bin already *is* a lock-in channel — so if several channels are willing to share one transform, one FFT demodulates all of them at once.

Frequency-division multiplexing (FDM)

Give each channel of interest its own distinct frequency — say 50, 51, 52 kHz on the same input wire, or on separate wires, it doesn't matter which. Take one windowed Fourier transform of a block of samples. Read each channel's amplitude and phase directly off its own bin of that one transform.



cost is one transform, ~O(N log N), regardless of how many bins you read out

FIG 4.1 One FFT, many channels read out from it. Adding a channel that already fits in the transform is essentially free — it's just another bin index to read.

Why a bin is exactly a lock-in channel

Write out one bin of a windowed DFT:

$$X[k] = \Sigma_n \; w[n] \cdot x[n] \cdot \exp(-j \cdot 2\pi \cdot k \cdot n/N) \qquad \text{for } n = 0 \ldots N-1$$

That is a correlation against a complex exponential at frequency `f_k = k·fs/N`, weighted by the window `w[n]`. Compare this to Chapter 2's mixer+filter: the exponential *is* the mixer, and the sum over the block *is* the low-pass filter — an FIR filter whose entire impulse response is the window shape, applied once per block. The bin index picks the reference frequency; the window shape is the filter; the block length plays the role of the time constant. It's the same operation as Chapter 2, computed a different way.

The condition that makes it exact: coherent sampling

This equivalence is only exact when a channel's frequency lands exactly on a bin centre:

> **f · N / fs must be an integer** — the tone must complete a whole number of cycles inside one block. Off by even half a bin and you get *scalloping loss* (the reading under-reports amplitude, up to ~3.9 dB worst case) and *spectral leakage* (energy smears into neighbouring bins) and a biased phase. Coherence checks this continuously and reports it in the status bar and the Debug tab — there's no clean after-the-fact correction, only picking `fs`, block size, and frequency so the condition holds up front.

Windows, and why Blackman-Harris is the default

The window is the filter, so its shape sets the trade-off between how sharply it separates neighbouring channels and how flat/accurate its response is:

WINDOW	MAIN LOBE WIDTH	HIGHEST SIDELOBE	NOTE
Rectangular	1×	−13 dB	sharpest, but bleeds badly into neighbours
Hann	2×	−32 dB	moderate
Blackman–Harris	4×	−92 dB	default — channels 10+ bins apart stay isolated
Flat-top	~10×	−90 dB	best amplitude accuracy, widest lobe

One more subtlety worth knowing: the window's main lobe sits around *every* spectral component, including DC. A tone parked on bin 1 or 2 sits inside the skirt of whatever DC offset is on the wire. Keep working tones at bin 8 or above with the default window.

Update rate is coupled to block size

One block of N samples takes `N/fs` seconds to fill. That's both the latency and — for disjoint blocks — the output update rate. Overlap processing (sliding the window by less than a full block; Coherence defaults to 50%) buys some of that back without changing the noise bandwidth, at proportional extra compute. But it's still fundamentally block-shaped: nothing comes out until enough samples have been collected to satisfy the bin-spacing and coherence requirements above — even for a channel that, on its own, wouldn't need that much data.

The streaming approach

A second engine, added specifically because block size ties resolution, DC-rejection, and update rate together whether you want that or not — including for a plain DC reading, which needs none of it.

NCO mixer + a running filter, per channel

Instead of collecting a block and transforming it once, each channel keeps two small pieces of state and updates them continuously, sample by sample (in practice, chunk by chunk — whatever size the acquisition delivers, however small):

1. A running reference phase (a *numerically-controlled oscillator*, NCO) — built straight from the absolute sample index, so it never needs resetting or resynchronizing.
2. A cascaded low-pass filter (Butterworth, run as second-order sections for numerical stability) applied to the mixed signal, whose internal state carries over from one chunk to the next — a genuinely continuous filter, not a new one built per call.

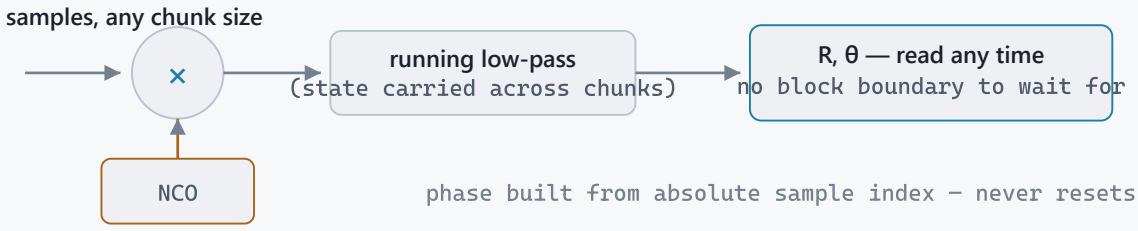


FIG 5.1 Continuous demodulation: mix, filter, read out — on whatever chunk of samples just arrived. No window, no block, nothing to wait for.

Time constant replaces block size

Each channel gets its own `time_constant_s`. The filter's cutoff is set from it, using the standard relation `f_cutoff = 1 / (2π·τ)`, exactly like the "time constant" dial on a bench lock-in. Shorter time constant: faster response, more noise gets through. Longer: cleaner, slower. This is the *same* speed-vs-noise trade-off every lock-in has — streaming doesn't remove it, it just lets you set it independently, per channel, completely decoupled from anyone else's setting and from how often you choose to read a value out.

No coherent-sampling requirement, at all

Because there's no window and no finite transform being truncated, there's no leakage or scalloping to avoid. A streaming channel works at literally any frequency, including exactly 0 Hz (DC) — the mixer for a DC channel is just multiplying by 1, and the same running filter smooths it directly.

One real subtlety, worth knowing if you ever touch this code: a real AC tone's energy splits evenly between +f and −f, so recovering true amplitude means doubling what the filter reports. A pure DC channel has no negative-frequency partner to account for, so it must *not* be doubled — the same special case the FFT engine already has for its own DC bin. Coherence handles this per channel automatically; it's mentioned here because it's the kind of thing that looks like a units bug if you don't know to expect it.

Update rate vs. block-gated: a picture

FIG 5.2 Same wall-clock span, same underlying noise averaging happening under the hood — but streaming has no hard boundary gating when a number is allowed to exist.

Choosing an engine

Both engines measure the same R/θ. The choice is about what you're willing to couple to what.

	TRADITIONAL (PER-CHANNEL)	FFT (BLOCK)	STREAMING (CONTINUOUS)
Cost per channel	O(N)	shared O(N log N) total	O(chunk) — cheap
Coherent sampling required?	no	yes — hard requirement	no
Update rate coupling	independent per channel	shared block_size/overlap for everyone	fully decoupled, per channel
DC / very low f	fine	needs a large block for bin separation from DC	trivial, fast, independent of other channels
Multiple tones on one wire	no — needs one physical input each	yes — that's the whole point (FDM)	yes, via per-channel filtering
Full diagnostic spectrum	no	yes, every block	not built (by design — that's the cost being avoided)
Best for	reference/legacy	many tones sharing one wire; you want to *see* the spectrum	fastest/smoothest read-out; DC; independent per-channel bandwidth

COOKBOOK

"I need a DC measurement at full speed"

1. Configure → **Engine: Streaming**.
2. On the channel: `frequency_hz = 0.0`.
3. Pick `time_constant_s` for the speed/noise trade-off you want — small (e.g. `0.001–0.005`) for the fastest response, larger if the reading is too noisy.
4. Sample rate and block size are irrelevant to this channel's speed now — there's no coherent-sampling condition to satisfy for DC, and no shared block gating the update.

The remaining ceiling is simply how often raw samples physically arrive from the driver (the DAQmx callback chunk size / ring-buffer drain rate) — not any DSP requirement. That's a tunable constructor parameter on the hardware backend today; ask if you want it exposed as a Configure-dialog control.

CHAPTER 7

Multiple cards, one acquisition

A chassis full of 4461s doesn't behave like one big card. This is the part that bites hardest if you don't know it up front.

DSA cards refuse outright to be combined into a single multi-device DAQmx task — not a degraded fallback, a hard error: "*One or more devices do not support multidevice tasks.*" The fix is one task per device, kept sample-aligned by wiring three things explicitly across every task:

1. **Reference clock.** Every task locks to the same chassis timebase (`PXIe_Clk100`, falling back to `PXI_Clk10`). Without this, each card's own oscillator free-runs and they drift apart.
2. **DSA sync pulse.** A shared clock alone isn't enough — remember Chapter 1's note about delta-sigma settling state. Every task gets `sync_pulse_min_delay_to_start` set to the *worst-case* settle time reported across every participating device, so nothing starts before the slowest card is actually ready.
3. **Start trigger.** Slave tasks are armed (started) first; the master starts *last* — its start is the digital edge that releases every armed slave at the same sample.

FIG 7.1 Calling `.start()` on a task with a digital-edge start trigger configured just arms it; it doesn't actually begin until the edge arrives. Starting the master last is what makes every card's first sample simultaneous.

This exact recipe applies on **both** sides of the acquisition — the AI backend and the AO stimulus generator independently, since AO channels spanning multiple devices hit the identical multi-device restriction. Whatever was actually applied is visible live in the app's **Debug** tab, per device, so a sync problem shows up as a message instead of quietly-wrong phase data.

Limits & quick reference

Hard limits to know before you configure anything

- **Nyquist:** no channel frequency may exceed `fs/2`. Both engines reject this at construction with the actual numbers, rather than an opaque index error.
- **FFT coherent sampling:** `frequency × block_size / fs` must be (near) an integer, or expect leakage/scalloping/phase bias. Streaming has no such requirement.
- **Sample rate ceiling in multi-device runs:** bounded by whichever participating card is slowest — Coherence names the limiting device if you ask for more than it can do.
- **DSA cards cannot share one multi-device task** — see Chapter 7. This is a hard DAQmx limitation of the 4431/4461 family, not a bug.
- **Driver buffer vs. the GIL:** the acquisition callback thread shares one Python interpreter with everything else (the DSP worker, the GUI). Too little onboard buffer turns an ordinary scheduling hiccup into a hard overrun. Coherence defaults to several seconds of buffer for exactly this reason.
- **GUI painting competes for the same CPU time as acquisition** on Windows — only the visible tab is repainted, and the diagnostic spectrum refreshes at a UI-friendly ~10 Hz regardless of the actual demodulation rate.

Quick reference: the numbers a new student needs

QUANTITY	FORMULA	NOTES
Nyquist limit	$f_s / 2$	hard ceiling on any measurable frequency
FFT bin spacing	f_s / N	N = block_size; also the FFT engine's frequency resolution
FFT update rate (no overlap)	f_s / N	with 50% overlap: ×2
Block duration	N / f_s	minimum latency of one FFT reading
Coherent frequency	$k \cdot f_s / N$	k integer — the only frequencies exactly on-bin
Streaming filter cutoff	$1 / (2\pi \cdot \tau)$	τ = time_constant_s, per channel
R (amplitude)	$2 \cdot \sqrt{(X^2+Y^2)}$	DC channel: no ×2 (no negative-frequency image)

Glossary

- **AI / AO** — analog input / analog output: the two physical operations a DAQ card performs.
- **DSA** — Dynamic Signal Acquisition: the 4431/4461 family's simultaneous-sampling, delta-sigma converter architecture.

- **Lock-in / synchronous demodulation** — recovering amplitude and phase at a known frequency by multiplying against a reference and low-pass filtering.
- **FDM** — frequency-division multiplexing: separating multiple signals sharing one wire by giving each its own frequency.
- **Coherent sampling** — a tone completing an exact integer number of cycles in one FFT block; required for leakage-free bin extraction.
- **NCO** — numerically-controlled oscillator: a reference sine/cosine generated digitally from a running phase, used by the streaming engine.
- **Time constant** — the low-pass filter's averaging window; sets the noise-vs-speed trade-off in every one of the three approaches.
- **Sync pulse / start trigger** — the multi-device mechanisms that keep several DSA cards' samples aligned to the same instant.

Coherence — FFT and streaming multichannel lock-in for NI-DAQmx hardware. See `docs/theory.md` and `docs/hardware-notes.md` in the repository for the same material in prose form, alongside the code itself.